

格式化读写文件

fprintf()函数

- printf ---- sprintf ---- fprintf
 - 变参函数，参数列表中，有 "...", 最后一个固定参数通常是一个模式描述串(包含格式匹配符)，函数实际调用时传递的参数个数、类型、顺序，由这个固定参数决定。

```
1 printf("hello");
2 printf("%s", "hello");
3 printf("%d = %d%c%d\n", 10+5, 10, '+', 5); ----> 屏幕
4
5 char buf[1024]; 内存空间 --- 缓冲区
6 sprintf(buf, "%d = %d%c%d\n", 10+5, 10, '+', 5) ----> buf 中
```

- 函数原型

```
1 #include <stdio.h>
2 int fprintf(FILE * stream, const char * format, ...); ----> 文件fp 中
```

- 测试

```
1 FILE* fp = fopen("abc", "w"); // 相对路径法
2 if (!fp) // NULL == fp
3 {
4     perror("fopen error");
5     return -1;
6 }
7
8 fprintf(fp, "%d%c%d=%d\n", 10, '+', 7, 10+7);
9
10 fclose(fp);
```

fscanf()函数

- scanf ---- sscanf ---- fscanf

```
1 int m;
2 scanf("%d", &m);          键盘 ----> m
3
4 char str[] = "98";
5 sscanf(str, "%d", &m);    str ----> m
6
7 FILE *fp = fopen("r");
8 fscanf(fp, "%d", &m);    fp指向的文件中 ----> m
```

- 函数原型

```
1 | int fscanf(FILE * stream, const char * format, ...);
```

- 测试:

```
1 | int a, b, c;  
2 | char ch;  
3 |  
4 | FILE* fp = fopen("abc", "r"); // 相对路径法  
5 | if (!fp) // NULL == fp  
6 | {  
7 |     perror("fopen error");  
8 |     return -1;  
9 | }  
10 |  
11 | (void)fscanf(fp, "%d%c%d=%d\n", &a, &ch, &b, &c);  
12 |  
13 | printf("a = %d\n", a);  
14 | printf("b = %d\n", b);  
15 | printf("c = %d\n", c);  
16 | printf("ch = %c\n", ch);  
17 |  
18 | printf("%d%c%d=%d\n", a, ch, b, c);  
19 |  
20 | fclose(fp);
```

格式化读写特性（扩展知识）

返回值

```
1 | int fprintf(FILE * stream, const char * format, ...);
```

- fprintf() 函数的返回值:
 - 成功：实际写入文件的字符个数。
 - 失败：-1

```
1 | int fscanf(FILE * stream, const char * format, ...);
```

- fscanf() 函数的返回值:
 - 成功：正确匹配的个数。
 - 失败：-1

fscanf 读取特性

1. 边界溢出问题。存储读取数据的空间，在使用之前，应该进行清空。否则会出现边界溢出异常。
 - 清空：memset(buf, 0, sizeof(buf));
2. fscanf 函数，每次在调用的同时，都会判断，下一次调用是否能成功匹配参2，如果不匹配提前结束读取文件（feof(fp) 为真）

练习：文件版排序

- 生成随机数，写入文件。将文件内乱序随机数读出，排好序再写回文件。

```
1 // 生成随机数，写入文件
2 void write_rand(void)
3 {
4     FILE* fp = fopen("test03.txt", "w"); // 相对路径法
5     if (!fp) // NULL == fp
6     {
7         perror("fopen error");
8         return -1;
9     }
10
11     // 播种随机数种子
12     srand(time(NULL));
13     for (size_t i = 0; i < 10; i++)
14     {
15         //int num = rand() % 100; // 0 -- 99
16         fprintf(fp, "%d\n", rand() % 100); // 将生成的随机数写入到文件。
17     }
18
19     fclose(fp);
20 }
21
22 void Bubblesort(int* src, int len)
23 {
24     for (int i = 0; i < len - 1; i++)
25     {
26         for (int j = 0; j < len - 1 - i; j++)
27         {
28             if (src[j] > src[j + 1])
29             {
30                 int temp = src[j];
31                 src[j] = src[j + 1];
32                 src[j + 1] = temp;
33             }
34         }
35     }
36 }
37
38 // 从文件中，读取随机数
39 void read_rand(void)
40 {
41     // 定义数组，存储 10 个随机数
42     int arr[10] = { 0 }, i = 0;
43
44     FILE* fp = fopen("test03.txt", "r"); // 相对路径法
45     if (!fp) // NULL == fp
46     {
47         perror("fopen error");
48         return -1;
49     }
50     // 循环读取文件内的随机数
51     while (1)
52     {
53         (void)fscanf(fp, "%d\n", &arr[i]);
```

```

54         i++;
55         if (feof(fp))    // 先存储，后判断，防止最后一个元素丢失。
56             break;
57     }
58     // 使用冒泡排序
59     BubbleSort(arr, sizeof(arr)/sizeof(arr[0]));
60
61     fclose(fp);    //关闭随机数据的文件
62
63     // 清空 随机数据的文件
64     fp = fopen("test03.txt", "w");
65     if (!fp)    // NULL == fp
66     {
67         perror("fopen error");
68         return -1;
69     }
70
71     for (size_t i = 0; i < 10; i++)
72     {
73         fprintf(fp, "%d\n", arr[i]);    // 将排好序的数组写入到文件。
74     }
75
76     fclose(fp);
77 }
78
79 int main(void)
80 {
81     write_rand();
82
83     getchar();    // 从键盘获取一个字符，如果用户不输入，就阻塞程序，不向下执行。
84
85     read_rand();
86
87     printf("-----finish\n");
88
89     system("pause");
90     return EXIT_SUCCESS;
91 }

```

按块读写文件

fgetc - fputc

fgets - fputs

fprintf - fscanf

以上3组函数，默认用来处理文本文件。

fwrite - fread 既可以处理文本文件，也可以处理二进制文件。

fwrite()函数

- 写出数据到文件中。

```

1  #include <stdio.h>
2  size_t fwrite(const void *ptr, size_t size, size_t nmemb, FILE *stream);
3      参1: 待写出的数据的首地址。
4      参2: 待写出数据的大小 （一次写多大）
5      参3: 写出的个数 （写多少次）    写出数据的总大小 = 参2 x 参3
6      参4: 文件fp
7      返回值:
8          成功: 永远返回参3。 通常调用函数时, 将参2传1, 参3代表实际写出的字节数。
9          失败: 0
10 // fwrite 函数, 写入数据到文件中时, 是按 二进制 写入。

```

- 测试

```

1  typedef struct student {
2      int age;
3      char name[10];
4      int num;
5  } stu_t;
6
7  // 按块写文件, fwrite
8  void write_struct(void)
9  {
10     // 定义结构体数组
11     stu_t stu[4] = {
12         18, "afei", 10,
13         20, "andy", 20,
14         30, "lily", 30,
15         16, "james", 40
16     };
17
18     FILE* fp = fopen("test04.txt", "w"); // 相对路径法
19     if (!fp) // NULL == fp
20     {
21         perror("fopen error");
22         return ;
23     }
24
25     int ret = fwrite(&stu[0], 1, sizeof(stu_t) * 4, fp);
26     if (ret == 0)
27     {
28         perror("fwrite error");
29         return ;
30     }
31     printf("ret = %d\n", ret);
32
33     fclose(fp);
34 }

```

fread()函数

- 从文件fp中读取数据。

```

1  size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream);
2      参1: 读取到的数据存放的地址。
3      参2: 一次读取数据的字节数 (一次读多大)
4      参3: 读多少次          读出数据的总大小 = 参2 x 参3
5      参4: 文件fp
6      返回值:
7          成功: 永远返回参3。 通常调用函数时, 将参2传1, 参3代表实际读出的字节数。
8          0 :  1) 读失败
9              2) 到达文件结尾 == feof(fp)为真

```

- 测试

```

1  typedef struct student {
2      int age;
3      char name[10];
4      int num;
5      // char addr[100];
6  } stu_t;
7  // 按块读文件, fread, 一次读一个 stu_t 元素
8  void read_struct(void)
9  {
10     FILE* fp = fopen("test04.txt", "r"); // 相对路径法
11     if (!fp) // NULL == fp
12     {
13         perror("fopen error");
14         return;
15     }
16     stu_t s1;
17
18     // 从文件中, 按块读取,
19     int ret = fread(&s1, 1, sizeof(stu_t), fp);
20     printf("ret = %d\n", ret);
21
22     printf("age=%d, name=%s, num=%d\n", s1.age, s1.name, s1.num);
23
24     fclose(fp);
25 }
26
27 // 按块读文件, fread, 一次读所有 stu_t 元素
28 void read_struct2(void)
29 {
30     FILE* fp = fopen("test04.txt", "r"); // 相对路径法
31     if (!fp) // NULL == fp
32     {
33         perror("fopen error");
34         return;
35     }
36
37     stu_t s1[10]; // stu_t *s1 = malloc(sizeof(stu_t) * 1024);
38     int i = 0;
39
40     // 从文件中, 循环按块读取,
41     while (1)
42     {
43         int ret = fread(&s1[i], 1, sizeof(stu_t), fp);
44         //if (ret == 0) // 效果一样。

```

```

45         if (feof(fp))    // 效果一样。
46         {
47             printf("读取到文件结尾\n");
48             break;
49         }
50         printf("age=%d, name=%s, num=%d\n", s1[i].age, s1[i].name,
s1[i].num);
51         i++;
52     }
53     fclose(fp);
54 }

```

练习：大文件拷贝

- 已知一个任意类型的文件，对该文件复制，产生一个相同的新文件。
- 实现思路：
 1. 打开两个文件，一个“r”，另一个“w”
 2. 从 r 中 fread，fwrite 写到 w 文件中
 3. 循环读，判断到达文件结尾，跳出循环。
 4. 关闭2个文件。
- 注意：在windows下，打开二进制文件（mp4、MP3、avi、jpg）读写，需要“b”。如：“rb”、“wb”

```

1  int main(void)
2  {
3      // 创建一个缓存区（内存空间），用来存储读到的数据。
4      char buf[4096] = { 0 };
5      int ret = 0;
6
7      FILE* rfp = fopen("C:\\Users\\afei\\Desktop\\TTTTTT\\11-午后回顾.avi",
"rb");
8      if (!rfp)
9      {
10         perror("fopen error");
11         return -1;
12     }
13     FILE* wfp = fopen("mycopy.avi", "wb");
14     if (!wfp)
15     {
16         perror("fopen error");
17         return -1;
18     }
19
20     // 循环从文件中读， 写到另一个文件中。
21     while (1)
22     {
23         ret = fread(buf, 1, sizeof(buf), rfp);
24         if (ret == 0)
25         {
26             break; // 读到文件末尾。
27         }
28         printf("ret = %d\n", ret);
29     }

```

```

30     // 没有读到文件末尾，将实际读到的数据，“原封不动的”写入 wfp 中。
31     fwrite(buf, 1, ret, wfp);
32 }
33
34 fclose(rfp);
35 fclose(wfp);
36
37 system("pause");
38 return EXIT_SUCCESS;
39 }

```

随机位置读写文件

- 强调：文件读写指针，在一个文件内，只有一个。读、写都使用这一个。

fseek

```

1  #include <stdio.h>
2  int fseek(FILE *stream, long offset, int whence); // 修改文件偏移量(文件读写指针)
3
4  参1: 文件fp
5  参2: 偏移量（矢量：正数向后，负数向前）
6  参3: 偏移的起始位置
7      SEEK_SET: 文件开头位置
8      SEEK_CUR: 当前位置
9      SEEK_END: 文件结尾位置
10 返回值: 成功: 0    失败: -1

```

ftell

```

1  long ftell(FILE *stream); // 获取文件偏移量（文件读写指针位置）
2  返回值: 从当前读写位置，到文件起始位置的偏移量。
3
4  // 借助 ftell(fp) + fseek(fp, -50, SEEK_END) 来获取文件大小。

```

rewind

```

1  void rewind(FILE *stream); // 回卷文件读写指针。将文件读写指针移动到起始位置。
2  // 相当于: fseek(fp, 0, SEEK_SET);

```

其他文件相关操作

Linux 和 Windows 文件区别

1. 对于二进制文件操作，Windows 下必须要使用“b”，Linux下二进制文件和 文本文件操作没区别。

2. windows下的回车换行 \r\n, 回车 \r, 换行 \n。Linux 下 回车换行 \n。

3. 对文件指针,

- 先写后读可以直接操作。windows 和 Linux 一致。
- 先读后写。Linux无序修改。Windows下需要在写操作之前, 添加 `fseek(fp, 0, SEEK_CUR)` 函数调用, 获取文件读写指针, 再来写。才能生效。

```
1 int main(int argc, char* argv[])
2 {
3     FILE* fp = fopen("test1.txt", "r+");
4
5     char buf[6] = { 0 };
6     char* ptr = fgets(buf, 6, fp);
7
8     printf("buf=%s, ptr=%s\n", ptr, buf);
9
10    fseek(fp, 0, SEEK_CUR); //获取文件读写指针。 如果没有这行。win下程序会崩溃。
11
12    int ret = fputs("AAAAA", fp);
13    printf("ret = %d\n", ret);
14
15    fclose(fp);
16
17    return 0;
18 }
```

获取文件状态

- `ftell(fp) + fseek(fp, 0, SEEK_END)` 可以获取文件大小。此种方法获取文件大小, 必须要打开文件, 文件打开、关闭操作, 对于系统而言, 系统资源消耗较大。

```
1 #include <sys/types.h>
2 #include <sys/stat.h>
3 int stat(const char *path, struct stat *buf); // status
4     参1: 文件访问路径
5     参2: 文件属性结构体指针 (传出参数: 函数调用结束时, 充当函数返回值)
6     返回值: 成功: 0    失败: -1
7 // 示例:
8 struct stat buf;
9 stat("待打开文件", &buf);
10 buf.st_size    获取文件的实际大小。
```

- 获取文件大小

```
1 struct stat buf;
2
3 int ret = stat("test06.txt", &buf); // buf传出参数
4 printf("ret = %d\n", ret);
5
6 printf("获取文件的大小为: %d\n", buf.st_size);
```

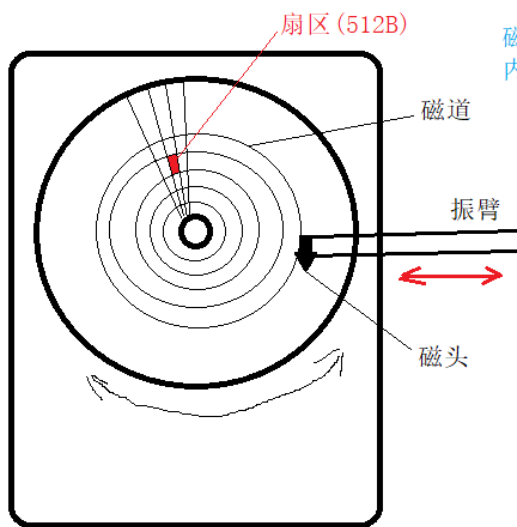
删除、重命名文件

```

1  int remove(const char *pathname); // 删除
2
3  int rename(const char *oldpath, const char *newpath); // 重命名
4
5
6  // 重命名:
7  //int ret = rename("test06.txt", "哼哼哈哈.txt");
8  //printf("ret = %d\n", ret);
9
10 // 删除文件
11 int ret = remove("哼哼哈哈.txt");
12 printf("ret = %d\n", ret);

```

缓冲区刷新



磁盘：物理操作。
内存：电子操作，接近于光速。

缓冲区：存在的意义，一次性物理访问磁盘过程中，尽可能多的读数据，保存在内存中。

预读入，缓输出。

- 标准输出 -- stdout -- 标准输出缓冲区。
 - 写给屏幕的数据，都是先存入缓冲区中，由缓冲区一次性刷新到物理设备（屏幕）
- 标准输入 --- stdin -- 标准输入缓冲区
 - 从键盘读取的数据，直接读到缓冲区，由缓冲区给程序提供数据。
- 缓冲机制：
 1. 行缓冲：遇到 `\n` 刷新缓冲区的数据到物理设备上。 `printf()`;
 2. 全缓冲：缓冲区存满，数据才刷新到物理设备上。文件。
 3. 无缓冲：缓冲区中只要有数据，立即刷新到物理设备。 `perror`
- 手动刷新缓冲区的方法：

```

1  #include <stdio.h>
2  int fflush(FILE *stream);
3      参：文件fp
4      返回值：成功：0    失败：-1

```

- 当文件关闭时，会强制刷新缓冲区，写入磁盘。—— 隐式回收
- 隐式回收：关闭文件。刷新缓冲区。释放 `malloc` 申请的内存。

```

1
2  int main(void)

```

```
3 {
4     FILE* fp = fopen("test10.txt", "w+");
5     if (!fp)
6     {
7         perror("fopen error");
8         return -1;
9     }
10    char ch = 0;
11
12    while (1)
13    {
14        (void)scanf("%c", &ch);
15        if (ch == ':')
16        {
17            break;
18        }
19        fputc(ch, fp);
20
21        // 手动刷新缓冲区，写入物理磁盘。
22        fflush(fp);
23    }
24    fclose(fp); // 当文件关闭时，会强制刷新缓冲区，写入磁盘。
25
26    system("pause");
27    return EXIT_SUCCESS;
28 }
```